Student Exchange | UML design atlas

Website + Node adapter + ASP.NET Core API + SQLite

A precise model of the system that exists today.

Prepared from the supplied UML PowerPoint PDF and the latest repository, including the newly added website. Each view has a distinct purpose, a boundary, and implementation evidence.

01 / Goals & structure

Actors, use cases, components, classes and data relationships

02 / Behavior & state

Request sequences, activities, sessions and marketplace UI

03 / Runtime & evidence

Deployment, integration gaps, permissions and traceability

Visual language: navy headings, pastel groups, explicit arrows, numbered views.

Status labels distinguish implemented behavior, browser previews and incomplete integration.

25 discrete diagrams, grounded in the repository and organized from goals to deployment.

Reading path & diagram index

Start with context; use the detailed views to answer one question at a time.

STRUCTURE / D01–D09

D01	System context
D02	Student & visitor use cases
D03	Administration use cases
D04	React component structure
D05	Node adapter structure
D06	Backend controller classes
D07	Request DTO inheritance
D08	Persistence class model
D09	Listing model & integration boundary

BEHAVIOR / D10–D18

D10	Login & cookie establishment
D11	Account registration
D12	Profile update & readback
D13	Admin account deletion
D14	Forwarding token renewal
D15	Logout & token revocation
D16	Browse, message & offer activity
D17	Token / target validation activity
D18	Attempt-window activity

STATE & RUNTIME / D19–D25

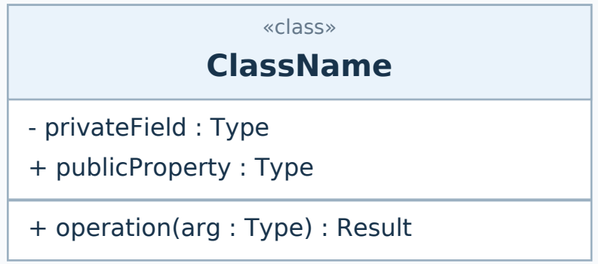
D19	Website session state machine
D20	Marketplace UI state machine
D21	Production deployment
D22	Package dependencies
D23	Startup & scheduled cleanup
D24	Listing API interactions
D25	Login communication diagram

Use the PDF to review, the editable PowerPoint to present, and the model + renderer to maintain.

Notation reference: supplied UML (1).pdf, pp. 7–43 and 57–81

UML notation & modeling rules

Conventions follow the lecture reference; architecture views are marked explicitly.



Class compartments

Name, attributes and operations; + public, - private. «static» and «interface» identify intent.

Association

A structural link; multiplicities appear at each end.

Dependency



Client uses supplier; arrow points toward supplier.

Generalization



Specialized type points to its parent type.

Realization



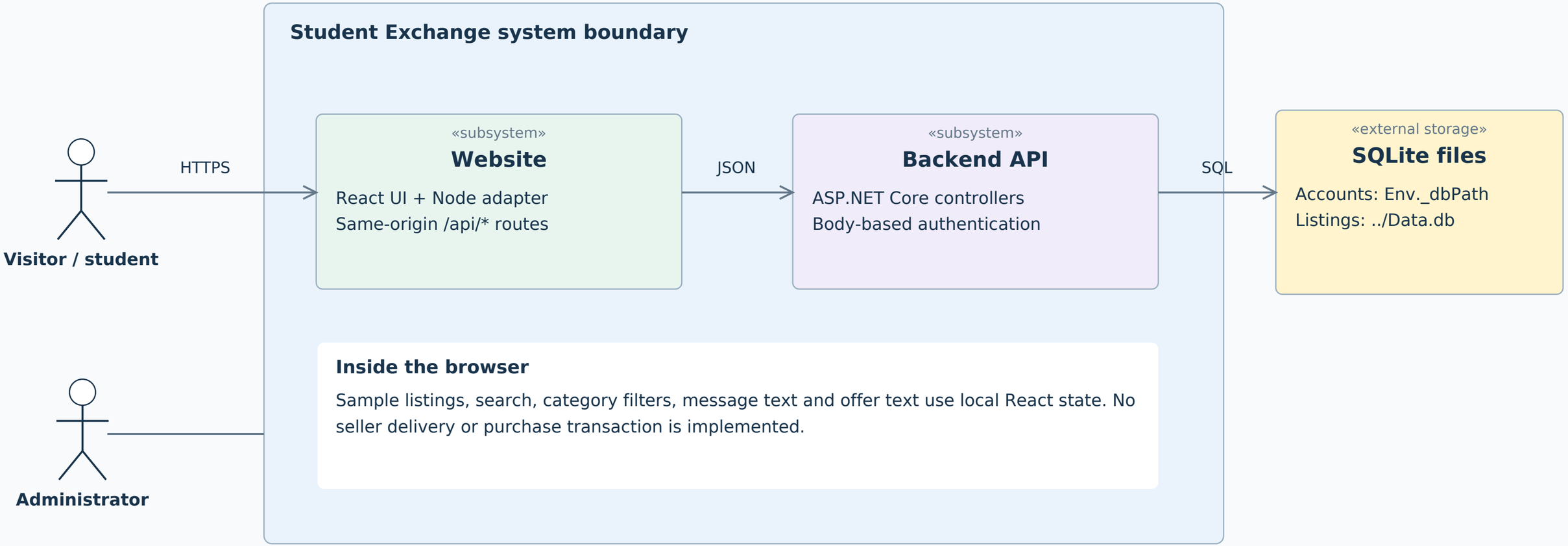
Implementation points to its interface contract.

Sequence: time descends; dashed arrows return. Activity: guards use [condition].

An association is not automatically composition; relational foreign keys do not establish object ownership.

System context

Architecture context • the website includes both browser UI and a server-side adapter.

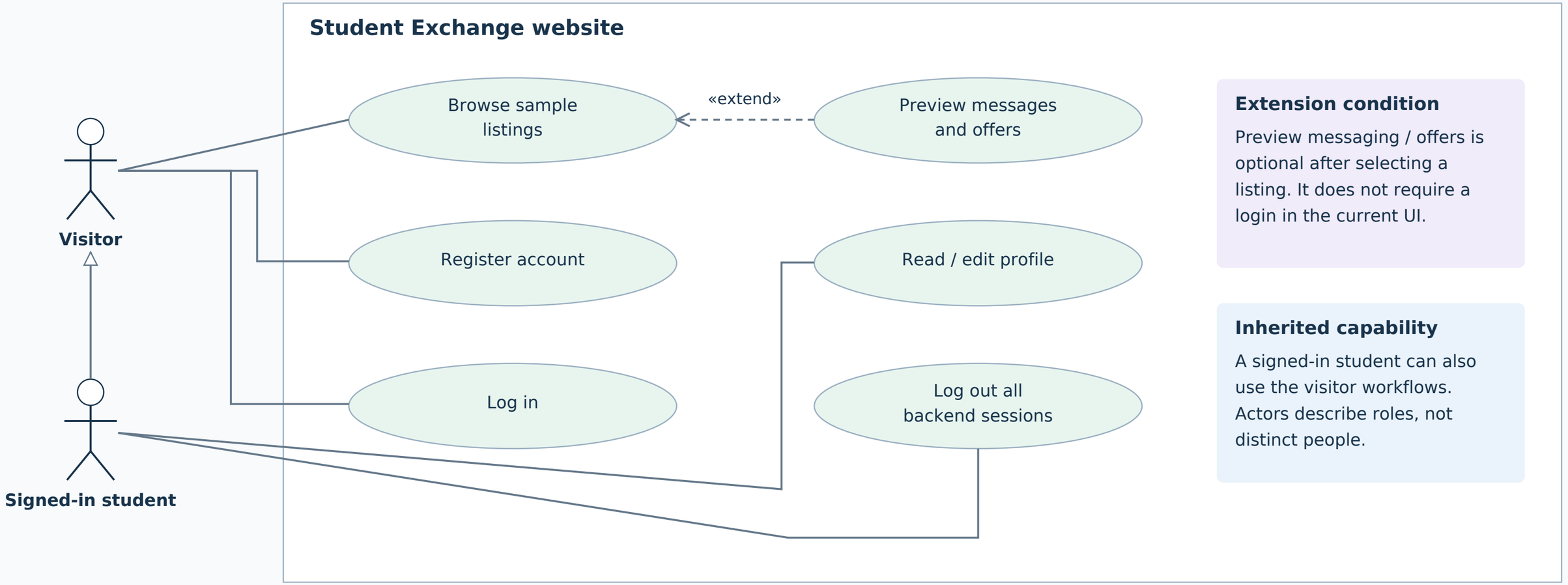


Account and admin workflows reach SQLite through the adapter; marketplace previews remain in the browser.

Evidence: Website/src/App.tsx; server/app.mjs, api.mjs; Backend/Program.cs; Controllers/Listing.cs

Student & visitor use cases

Use-case model • boundary covers the website and its account adapter.



Extension condition

Preview messaging / offers is optional after selecting a listing. It does not require a login in the current UI.

Inherited capability

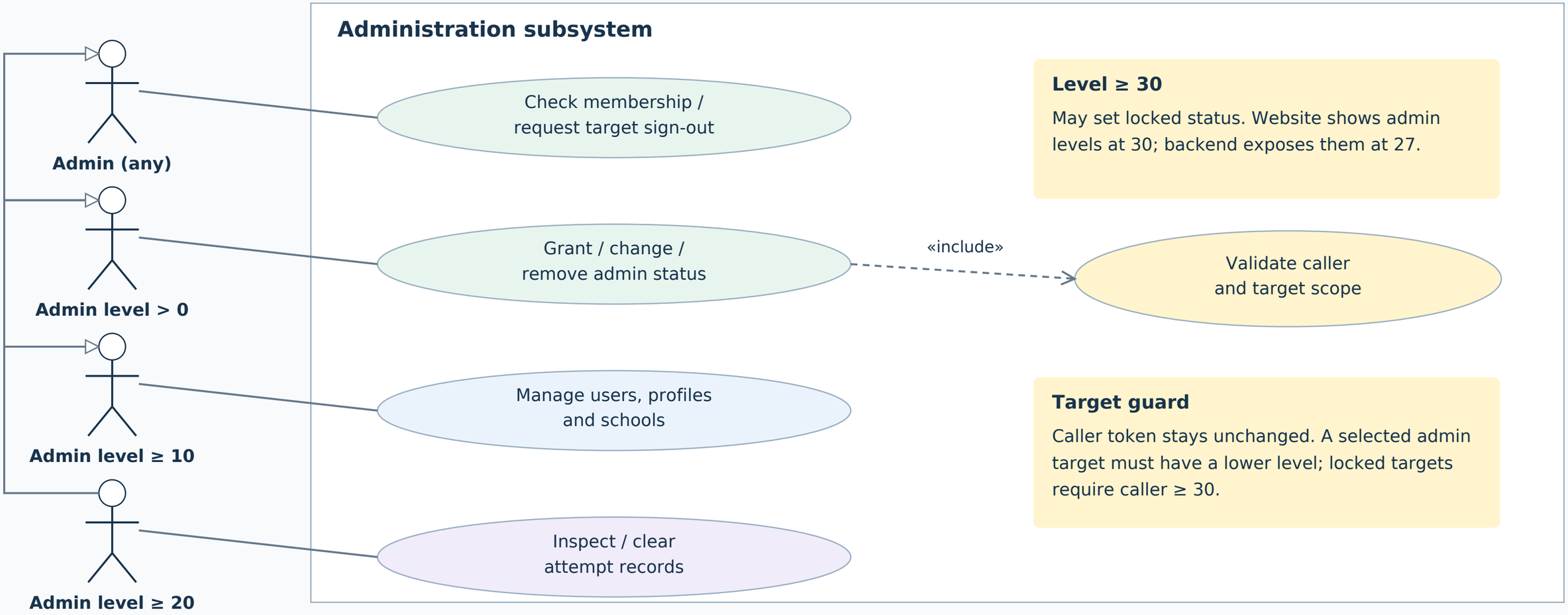
A signed-in student can also use the visitor workflows. Actors describe roles, not distinct people.

Registering creates an account; a separate login establishes the website session.

Evidence: Website/src/App.tsx, hooks/useAccount.ts, api/accounts.ts; server/api.mjs

Administration use cases

Use-case model • specialized admin roles inherit the capabilities of lower-level roles.

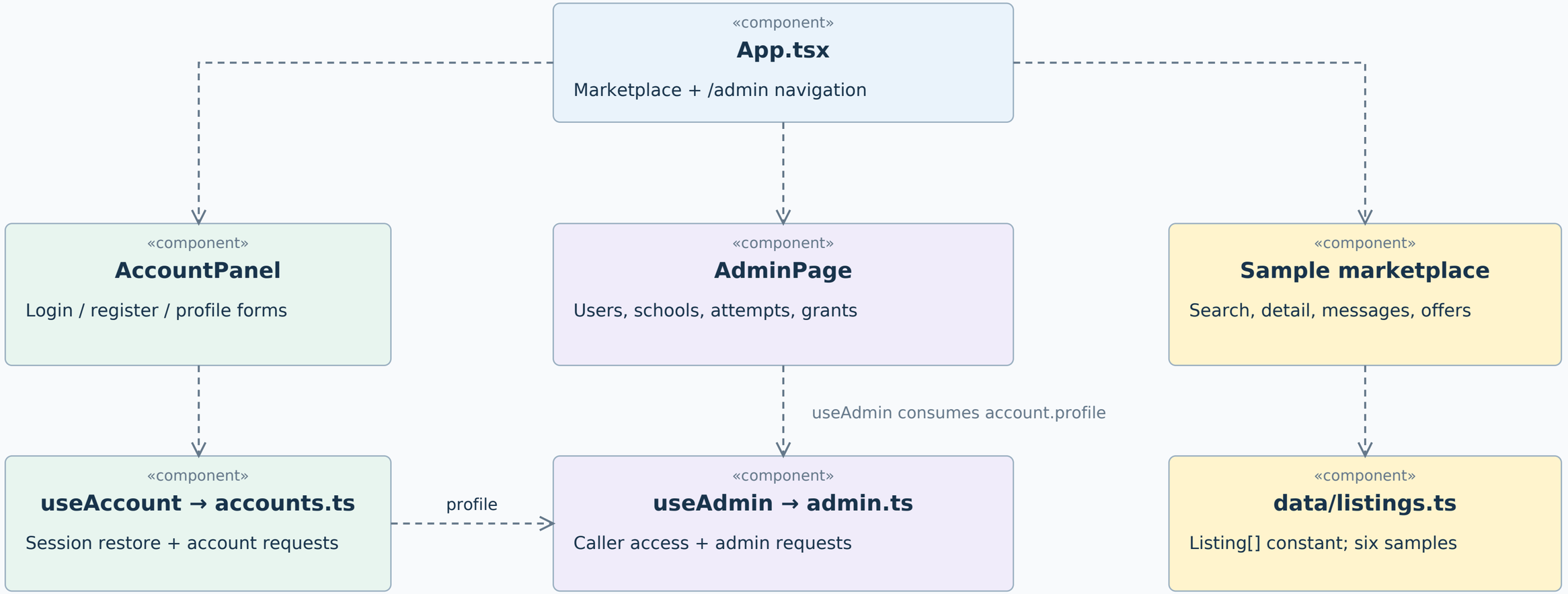


Every admin operation refreshes membership; target-user restrictions are checked again by the backend.

Evidence: Website/server/admin.mjs, api.mjs; Backend/Controllers/Admin.cs, Blocked.cs, Register.cs, Schools.cs

React component structure

UML component view • React functions are components, not invented object-oriented classes.

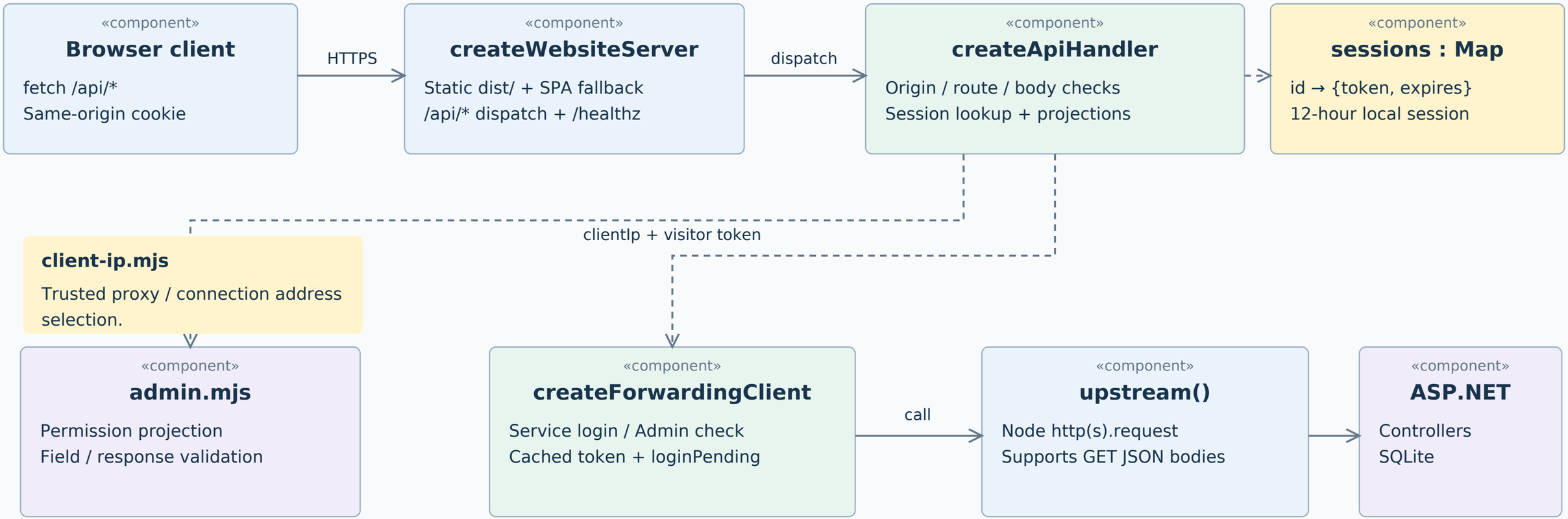


The hooks own account and admin state; App owns marketplace previews and page selection.

Evidence: Website/src/App.tsx, components/*, hooks/useAccount.ts, hooks/useAdmin.ts, api/*

Node adapter component structure

UML component view • browser-safe contracts are translated into backend body credentials.

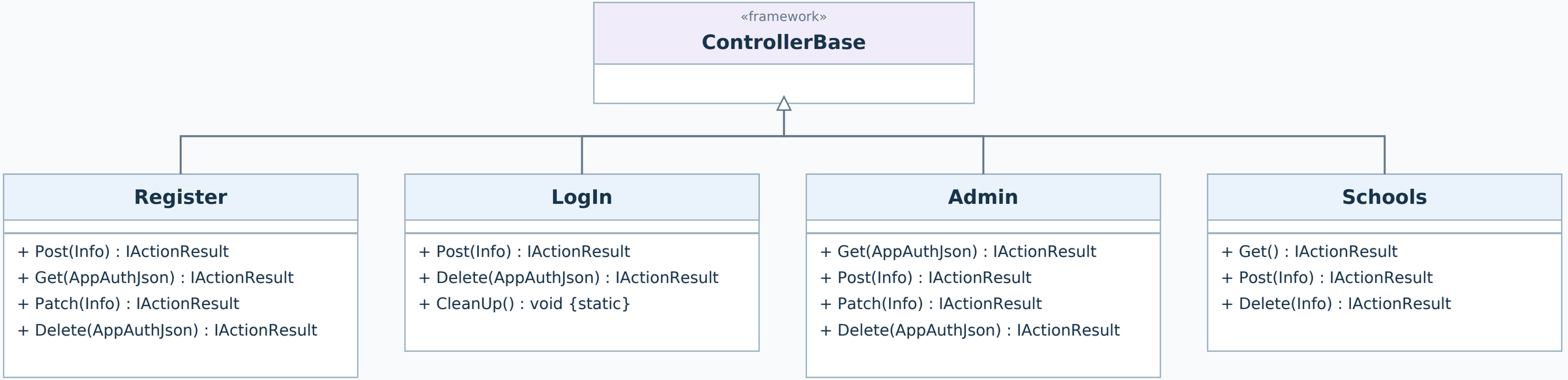


Opaque cookie IDs stay in the browser; backend user and service tokens stay in Node memory.

Evidence: Website/server/app.mjs, api.mjs, admin.mjs, client-ip.mjs; src/api/accounts.ts

Backend controller classes

UML class view • abbreviated public operations; all controllers inherit ControllerBase.



Users : ControllerBase

+ Get(AppAuthJson)
Also user locks + attempts

Blocked : ControllerBase

+ Get(AppAuthJson)
+ Delete(Info)

ListingController

GetListings / CreateListing
DeleteListing; own SQL path

Shared helpers

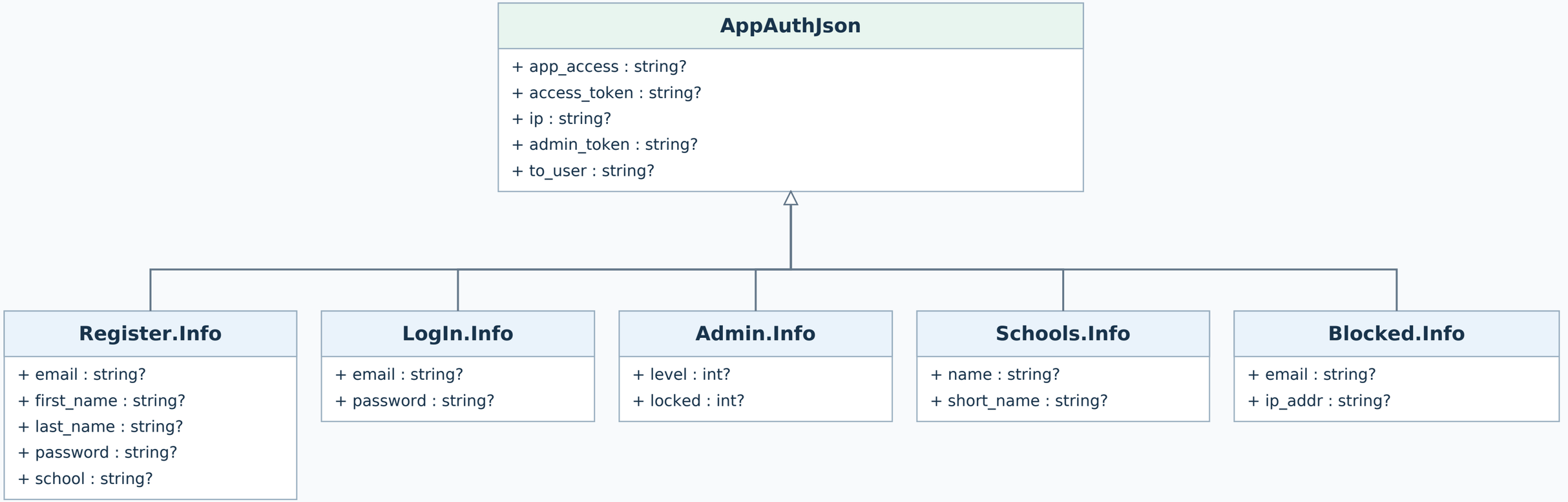
AppAuthJson, IP, Users, Sql
Env, Utilz, encryption

Controllers call shared helpers directly; the code does not implement a separate service/repository layer.

Evidence: Backend/Controllers/*.cs; Components/Users.cs, Sql.cs; Listing.cs

Request DTO inheritance

UML class view • nullable fields reflect the actual request types.



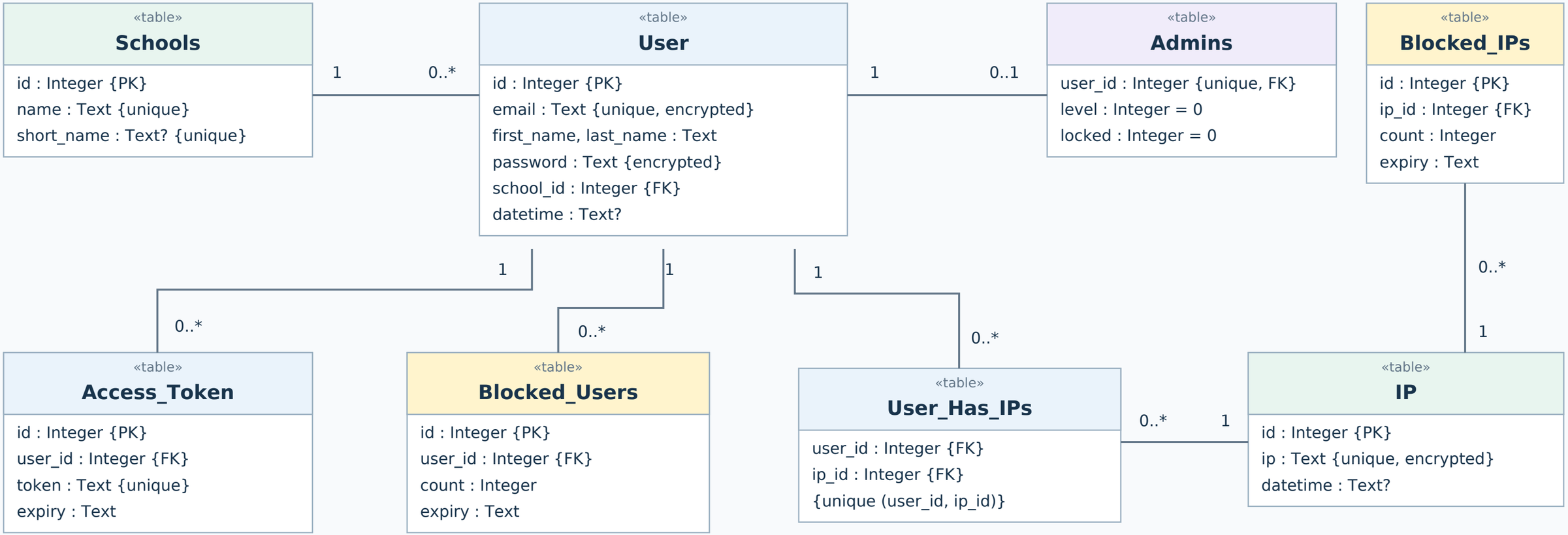
Info types inherit the shared credential and forwarding fields.

admin_token is a login-issued admin token for IP forwarding; access_token identifies the acting user.

Evidence: Backend/Components/AppAuthJson.cs; Controllers/{Register,LogIn,Admin,Schools,Blocked,Listing}.cs

Persistence class model

UML class view of database tables • multiplicities describe valid schema relationships.

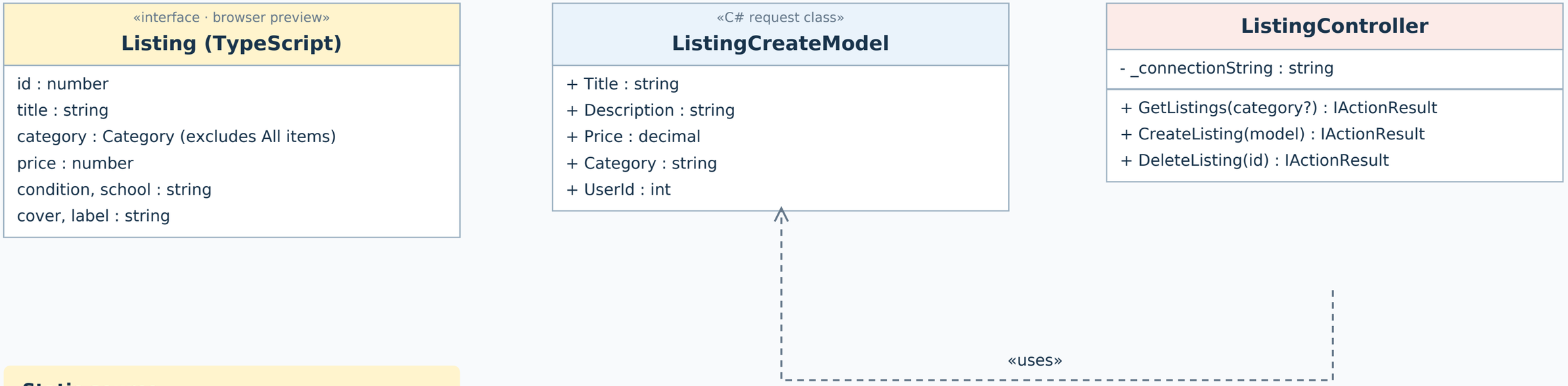


The schema declares delete rules; actual cascade behavior depends on SQLite foreign-key enforcement.

Evidence: DB_Schema.txt; Backend/Components/Sql.cs; AppAuthJson.cs; not an ORM class model

Listing model & integration boundary

UML class / component view • the browser listing and API listing are different contracts.



Static source

Six illustrative records. Messages and offer strings are held in App state; no seller account or persisted conversation.

API data expectation

Listings: id, title, description, price, category, user_id, datetime; joined to User. Table inferred from SQL, not declared DDL.

Separate connection

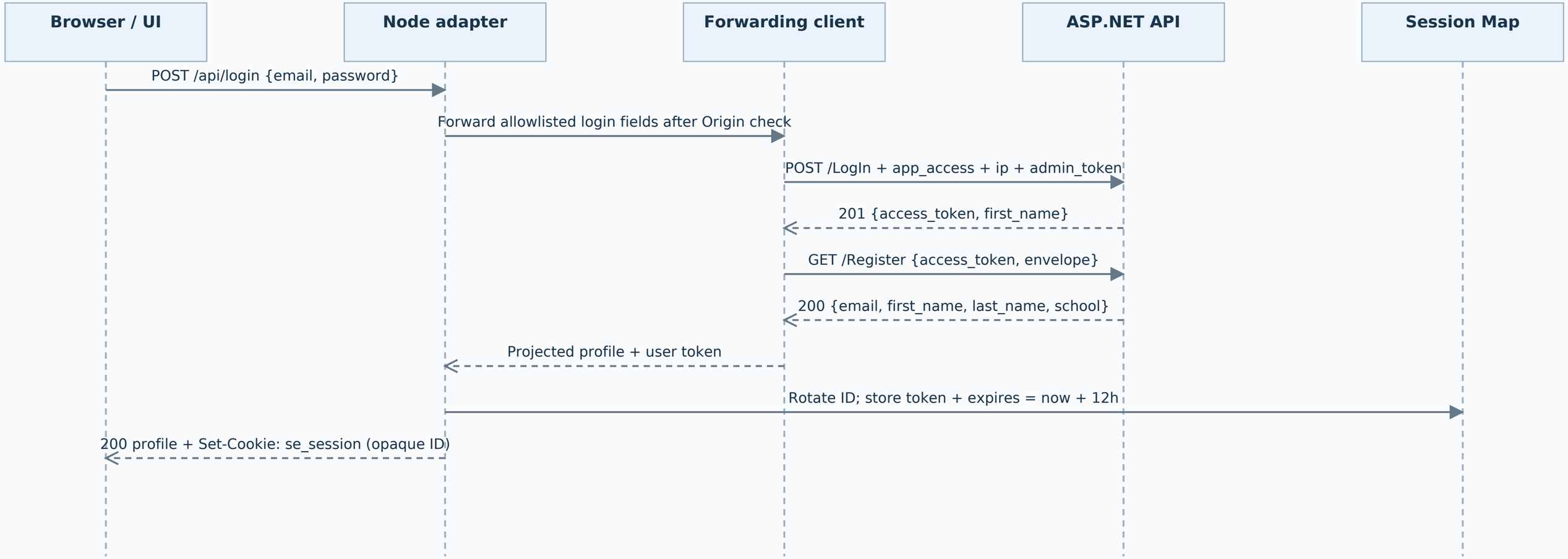
Data Source=../Data.db; no shared AppAuthJson checks in listing actions.

The website does not call ListingController; Listings is absent from the supplied database schema.

Evidence: Website/src/data/listings.ts, App.tsx, server/api.mjs; Backend/Controllers/Listing.cs; DB_Schema.txt

Login & website session establishment

Sequence • configured forwarding service token is assumed ready; D14 details its lifecycle.

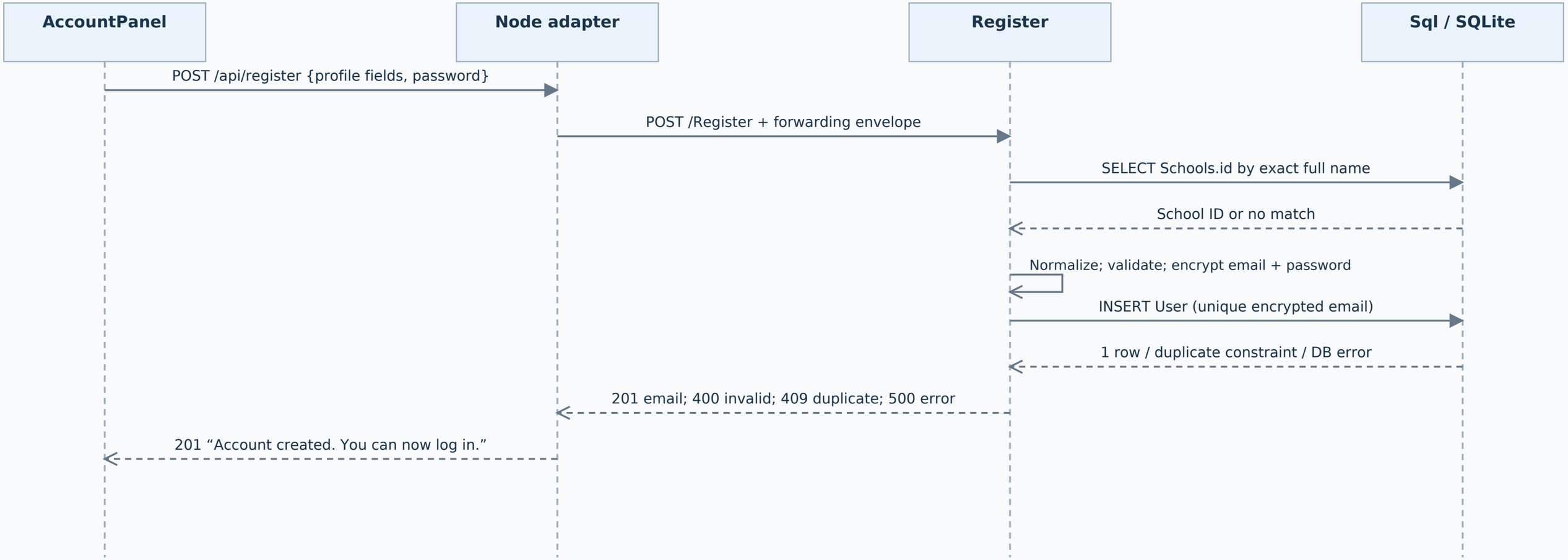


The browser receives a profile and an HttpOnly cookie, never the backend access token.

Evidence: Website/server/api.mjs: /api/login, publicProfile, setCookie; hooks/useAccount.ts

Registration & validation

Sequence • registration is independent of login; no website session is created.

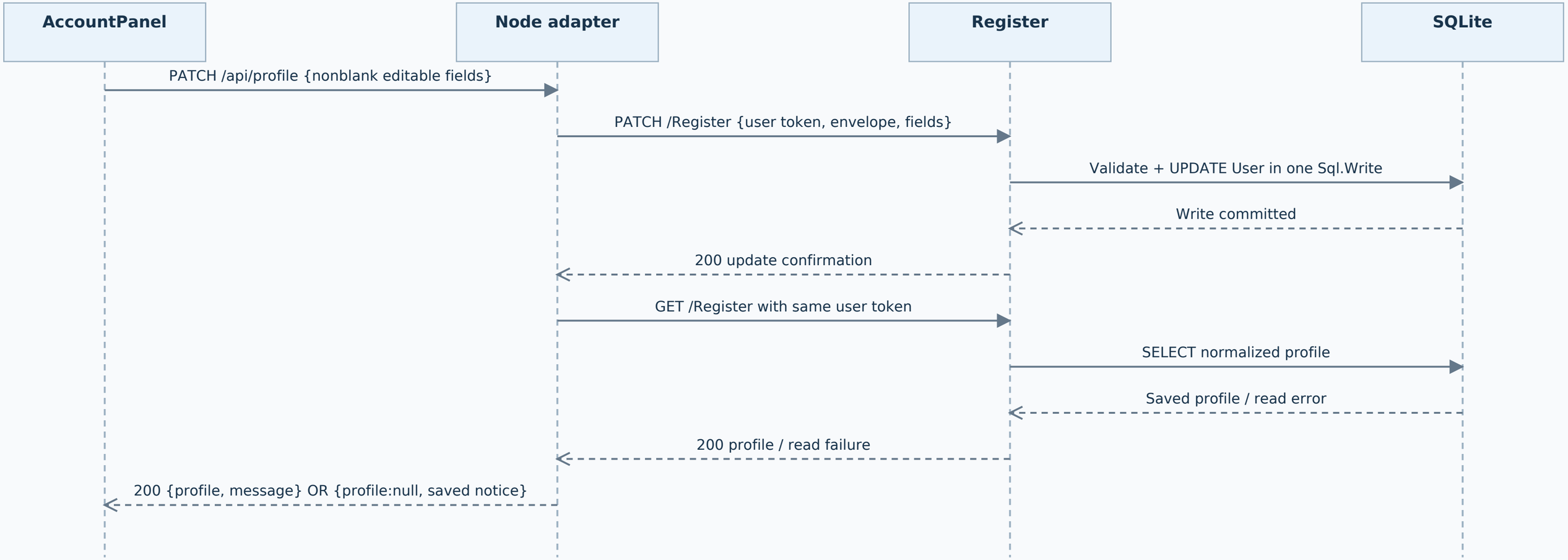


Validation includes an existing school, valid names/email, and an 8–50 character password.

Evidence: Website/server/api.mjs: /api/register; Backend/Controllers/Register.cs: Post, ValidateFormatInfo

Profile update & readback

Sequence • happy path followed by an explicit readback-failure alternative.

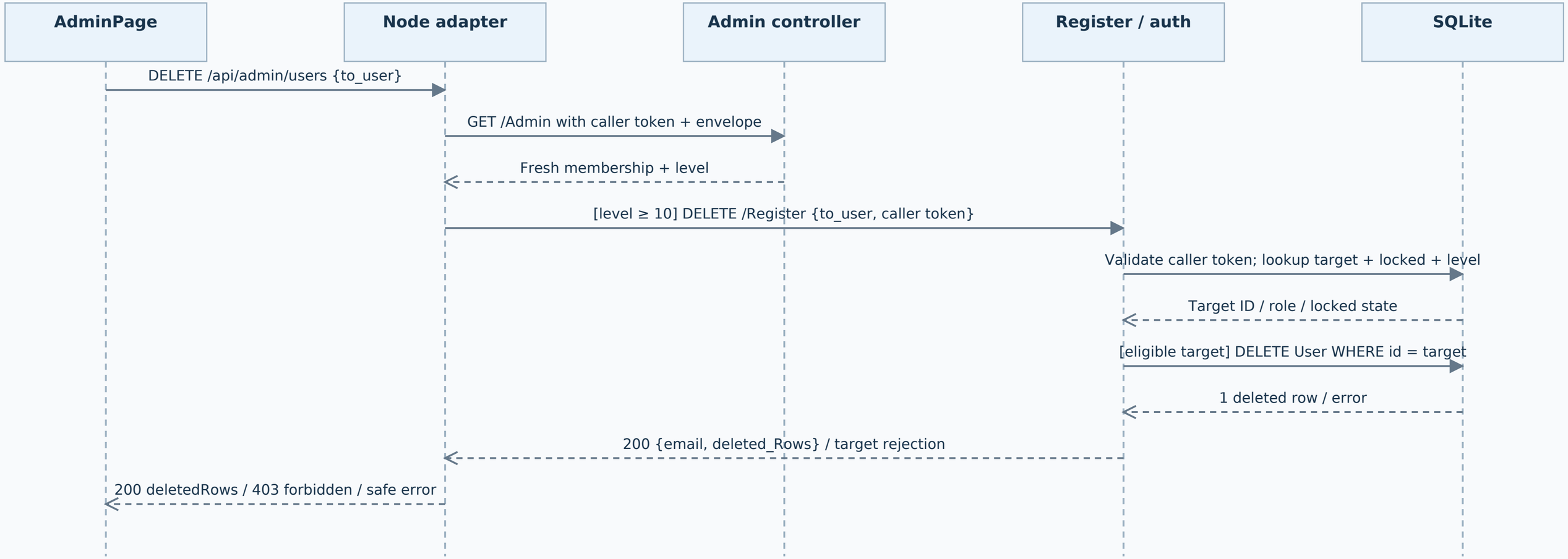


Once PATCH succeeds, a failed follow-up read does not trigger a second write.

Evidence: Website/server/api.mjs: /api/profile PATCH; hooks/useAccount.ts: update; Backend/Register.cs: Patch

Admin account deletion

Sequence • target is selected with to_user; caller token remains the administrator’s token.

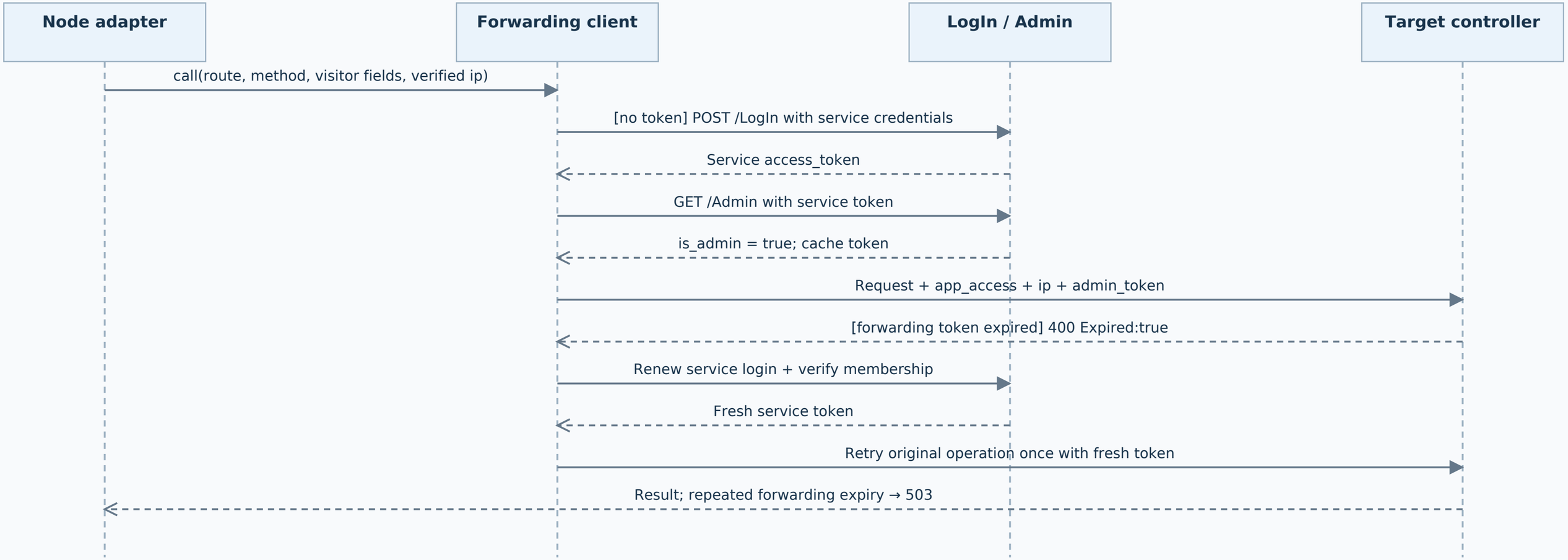


An admin target must have a lower level; locked targets require caller level ≥ 30.

Evidence: Website/server/api.mjs: admin/users; Backend/AppAuthJson.cs: ValidateToken; Register.cs: Delete

Forwarding service token renewal

Sequence • one service account token per Node adapter; loginPending coalesces concurrent renewal.

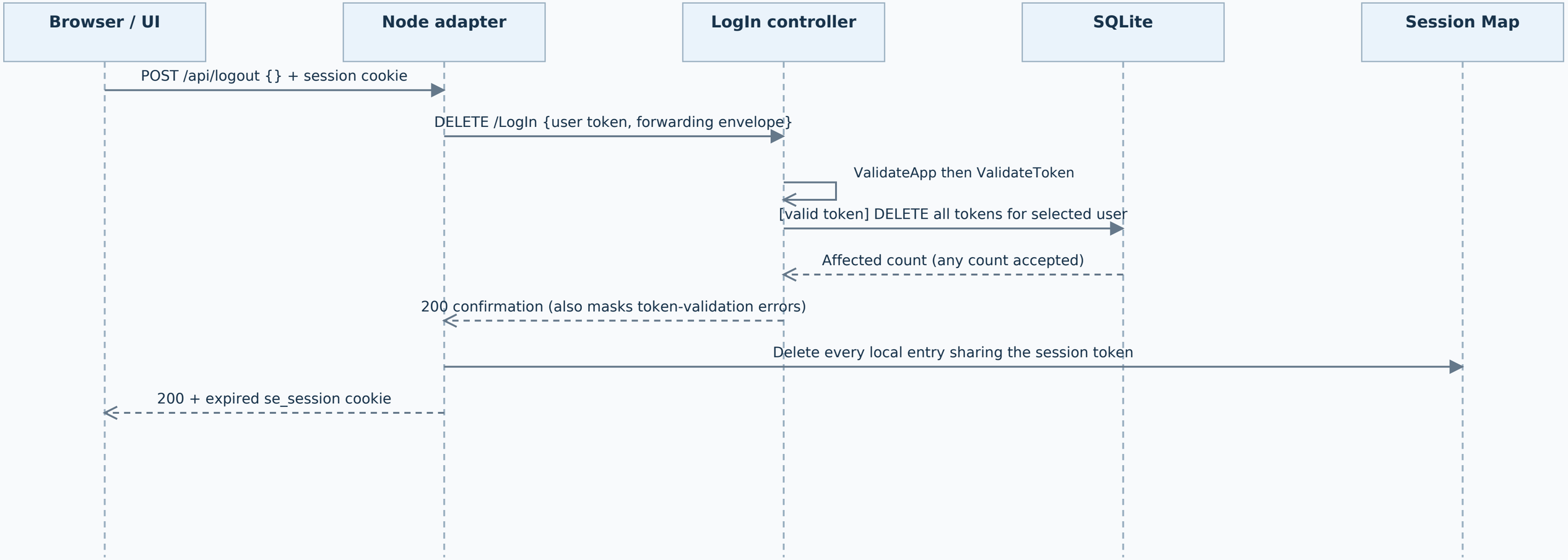


Retry occurs only for forwarding expiry, which the backend rejects before executing the target action.

Evidence: Website/server/api.mjs: createForwardingClient, upstream; Backend/AppAuthJson.cs: ValidateApp

Logout & revocation

Sequence • shown with a valid website session; absent sessions return an idempotent 200.

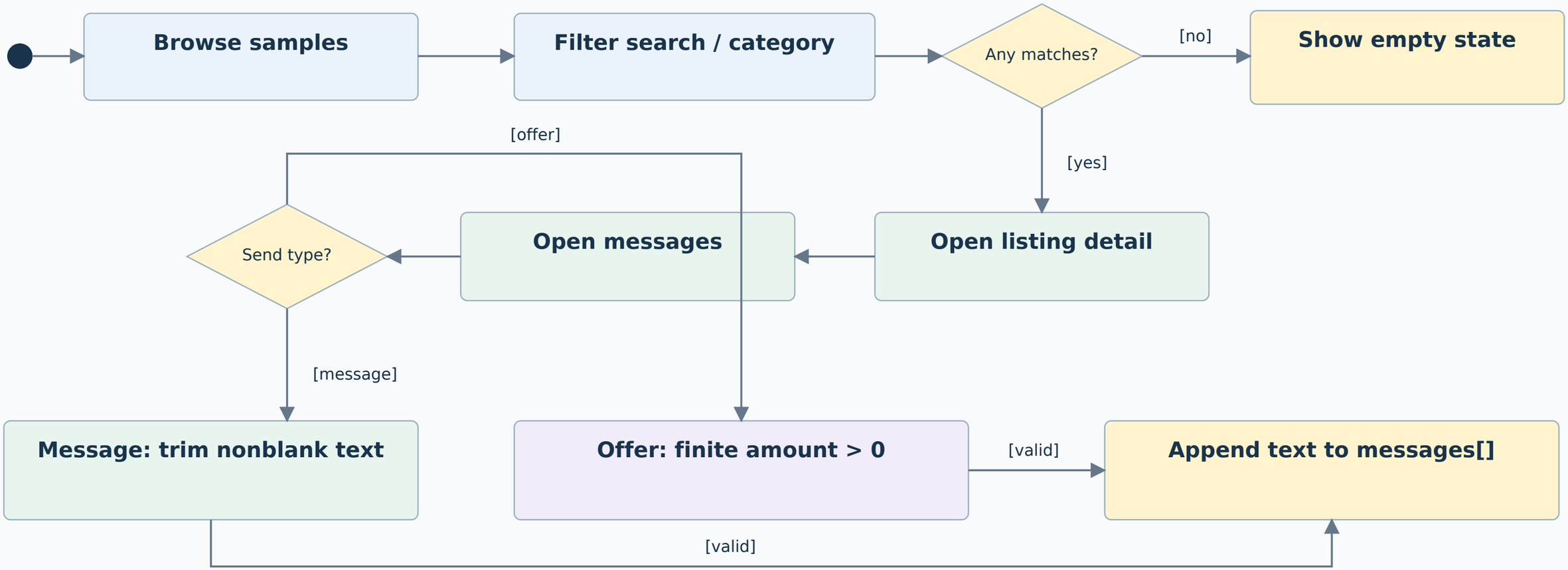


The backend’s 200 does not prove token deletion; the adapter still clears matching local sessions.

Evidence: Website/server/api.mjs: /api/logout; Backend/Controllers/LogIn.cs: Delete

Browse, message & offer activity

UML activity view • all actions shown occur in browser memory.

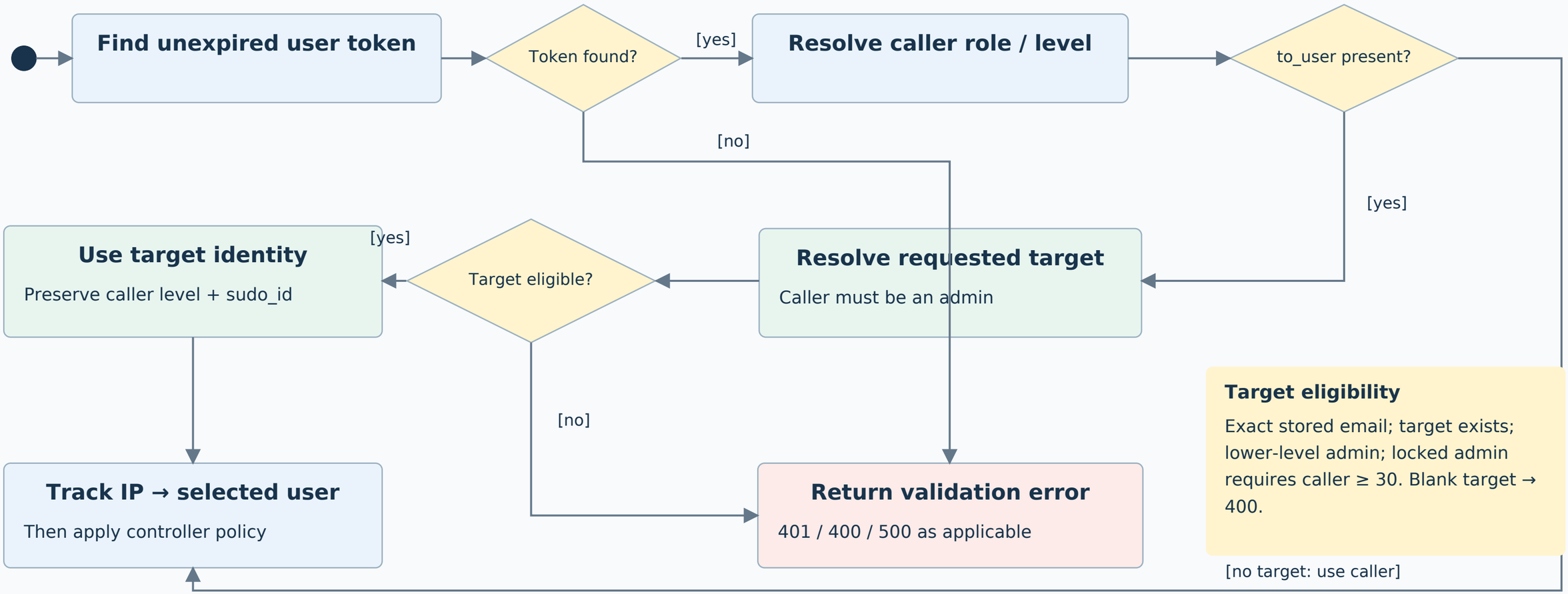


Message and offer submissions append local text; there is no network delivery or durable transaction.

Evidence: Website/src/App.tsx: visibleListings and message / offer form handlers

Token & target validation activity

UML activity view • ValidateApp has already passed; route-specific checks follow this activity.

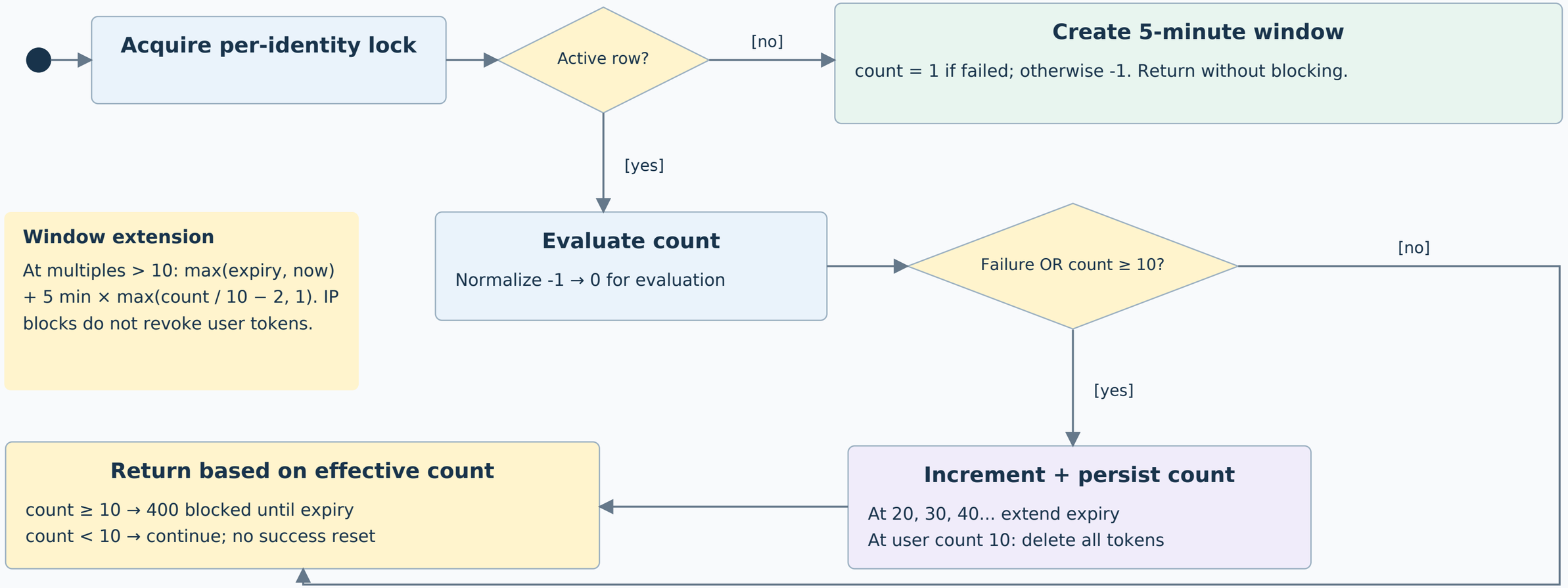


Forwarding preserves caller adminLvl and sudo_id while selecting the target user.id and isAdmin.

Evidence: Backend/Components/AppAuthJson.cs: ValidateToken; Register.cs, Admin.cs, Blocked.cs

Attempt-window activity

UML activity view • common pattern in Users and IP; known-user path shown.

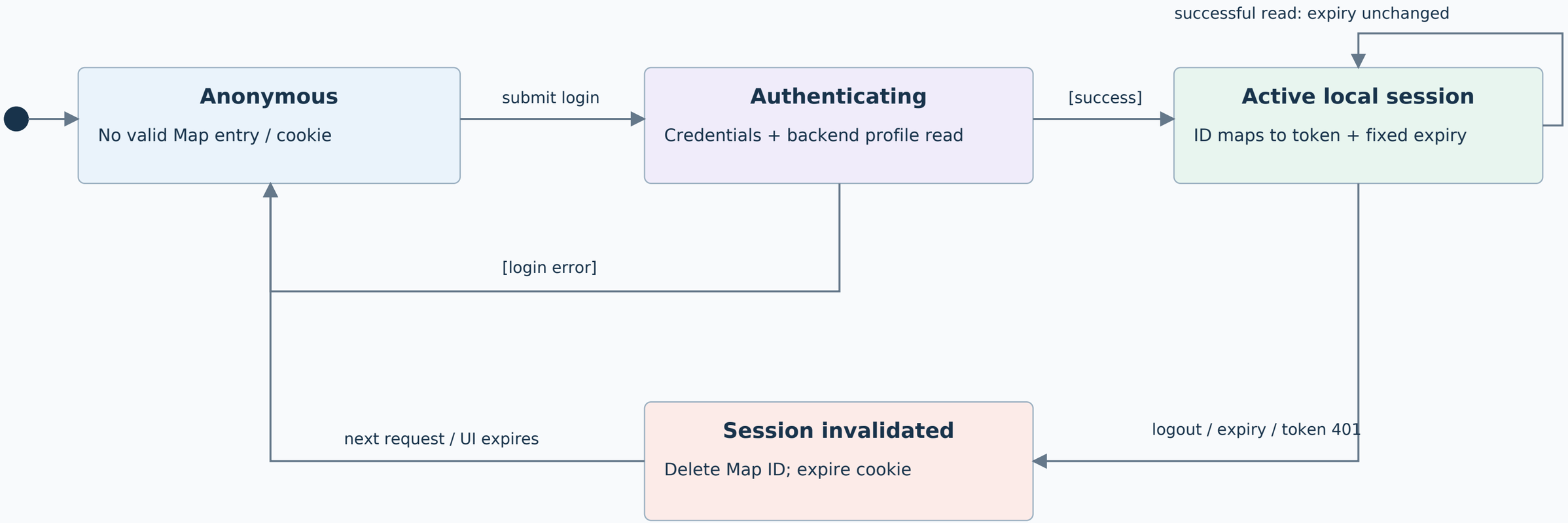


Count -1 is a successful observation, not a block. Active counts ≥ 10 block access.

Evidence: Backend/Components/Users.cs: IncInvalidForUser; IP.cs: IncInvalidForIP; Env.cs

Website session state machine

UML state machine • adapter session validity and backend token validity are separate.



Restart behavior

A process restart loses every Map entry. A remaining browser cookie cannot restore its backend token.

Transient failure

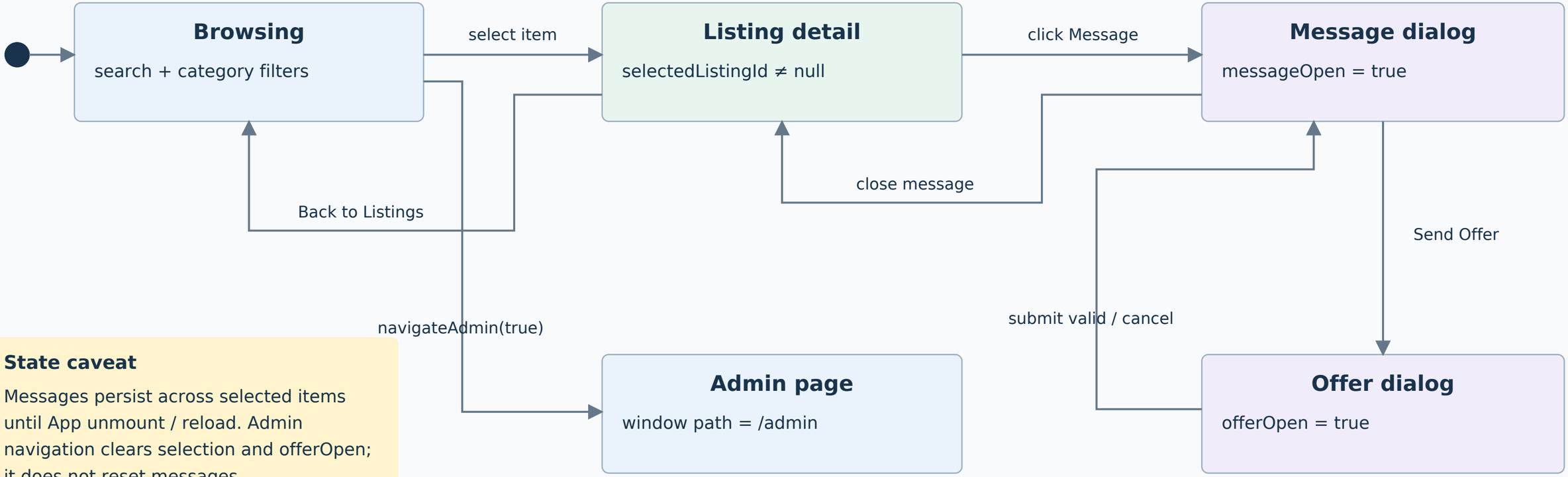
502/504 does not by itself delete a session.

Adapter sessions expire after 12 hours, are not sliding, and disappear on a Node process restart.

Evidence: Website/server/api.mjs: SESSION_LIFETIME, sessions Map, sessionFor, login/logout, catch handler

Marketplace UI state machine

UML state machine • a simplified projection of App state, not a persisted business lifecycle.



State caveat

Messages persist across selected items until App unmount / reload. Admin navigation clears selection and offerOpen; it does not reset messages.

No business transaction state

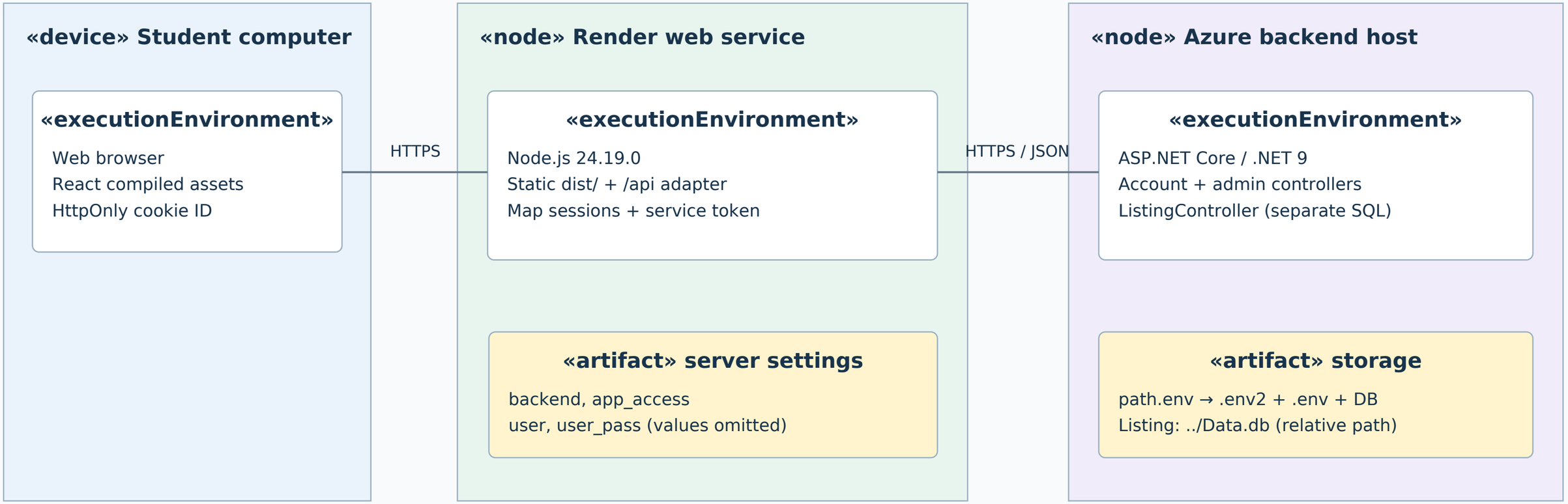
There is no persisted offer acceptance, payment, order, delivery or seller-contact state machine.

Offers are strings in a shared messages array; the current code does not isolate conversations by listing.

Evidence: Website/src/App.tsx: selectedListingId, messageOpen, offerOpen, messages, navigateAdmin

Production deployment

UML deployment view • derived from render.yaml and source configuration, not a verified live topology.



Build: npm ci + npm run build → dist/

Process health: GET /healthz

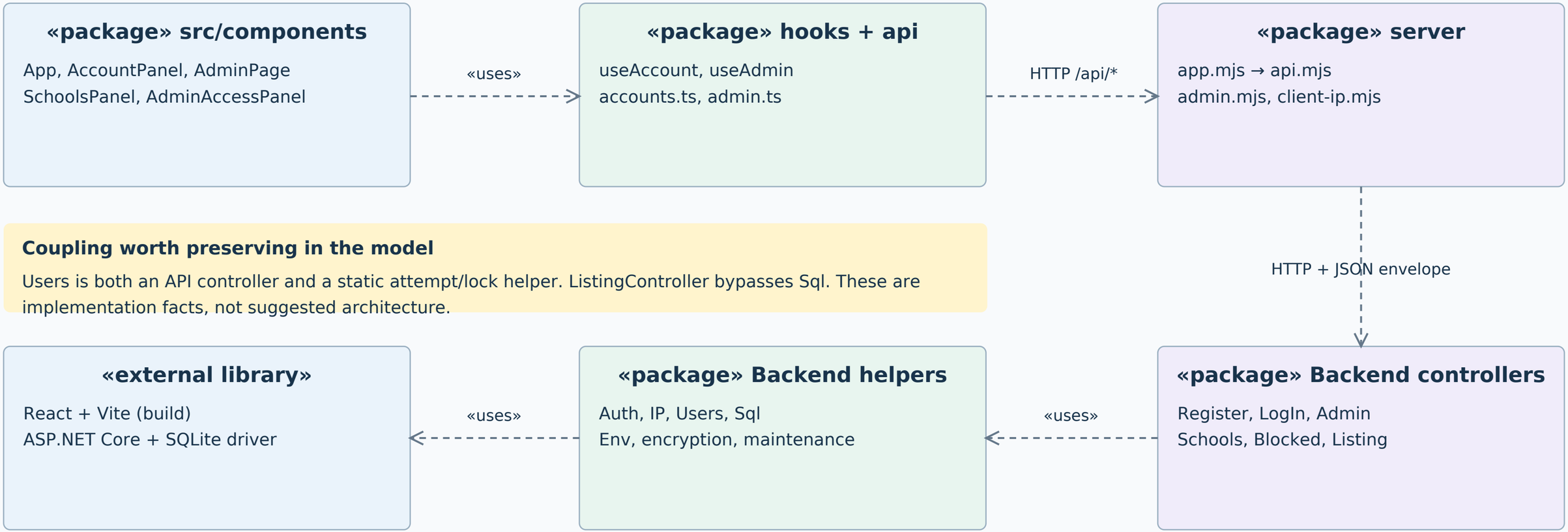
Configured replicas: 1

Render is configured for one Node instance; in-memory sessions do not support independent replicas.

Evidence: Website/render.yaml, server/index.mjs, app.mjs; Backend/Program.cs, Env.cs, Sql.cs, Listing.cs

Package dependencies & architectural style

UML package view • a layered request path with direct helper coupling inside the backend.



Coupling worth preserving in the model

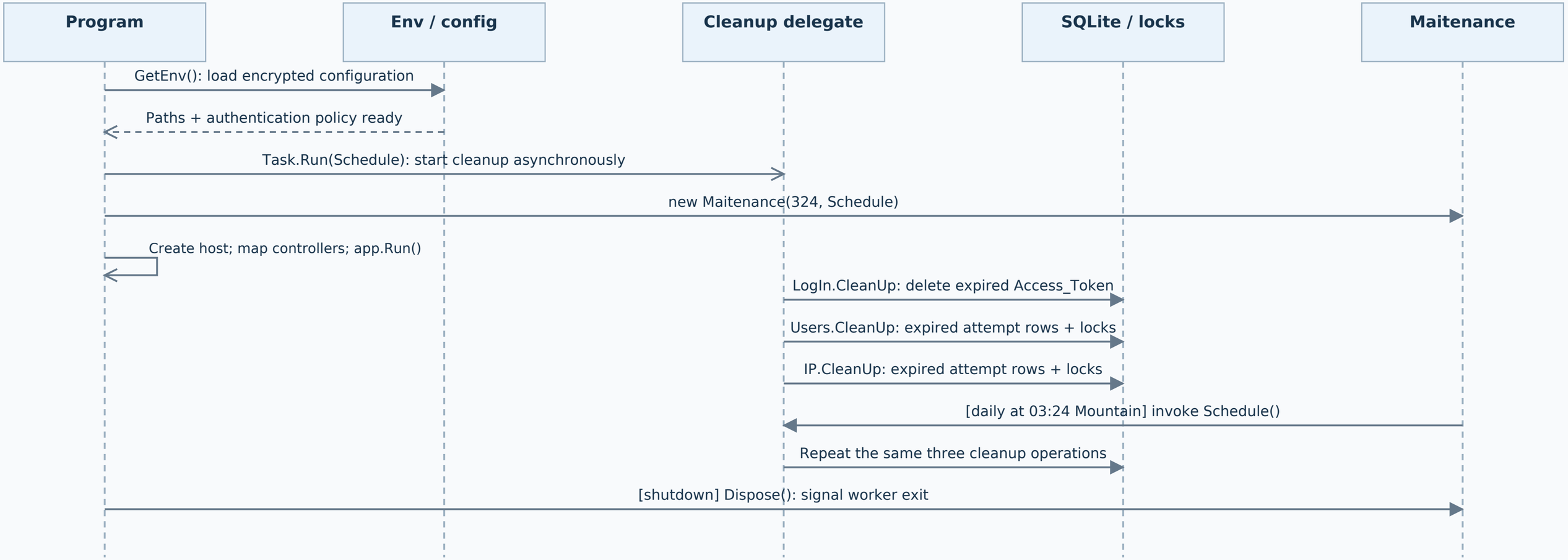
Users is both an API controller and a static attempt/lock helper. ListingController bypasses Sql. These are implementation facts, not suggested architecture.

The current system has a browser/adapter/API split; a full MVC view layer or DDD service layer is not implemented.

Evidence: Website/src and server imports; Backend/Program.cs; controller/helper source dependencies

Startup & daily cleanup

Sequence • one possible interleaving; the startup cleanup task is not awaited.

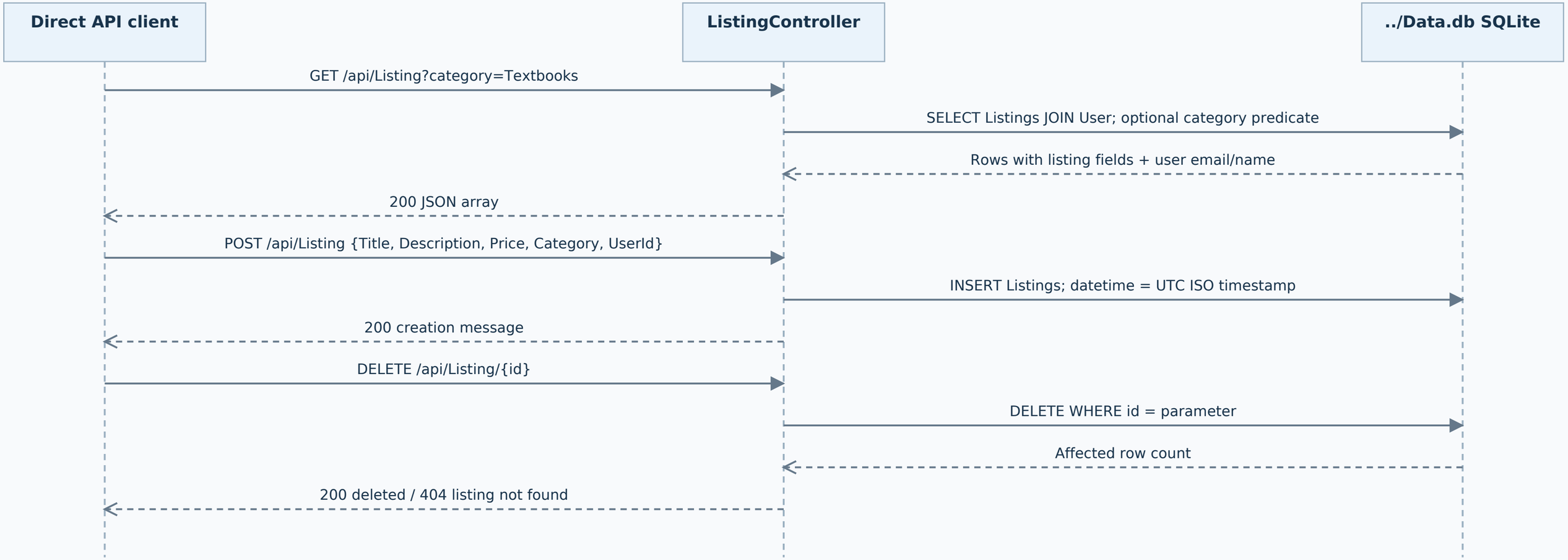


Startup cleanup can overlap host startup; the scheduled worker repeats cleanup daily at 03:24 Mountain.

Evidence: Backend/Program.cs; Components/Env.cs, Maitenance.cs, Users.cs, IP.cs; LogIn.cs

Listing API interactions

Sequence • direct API client; this path is not invoked by the current website adapter.

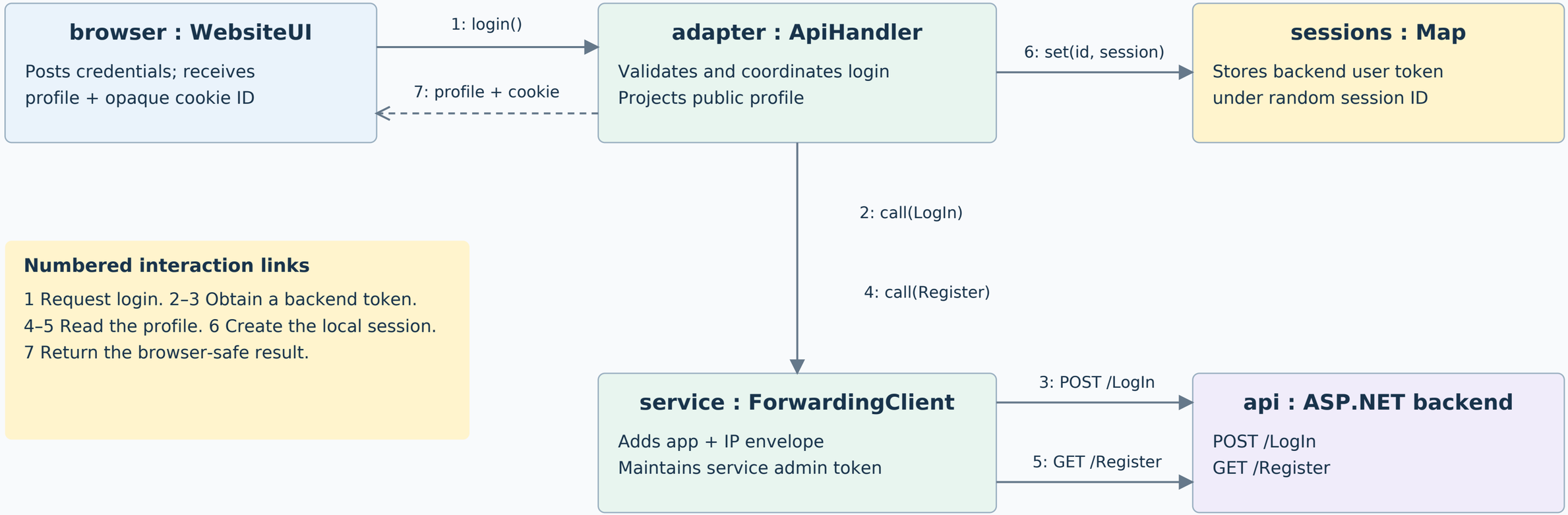


Listing actions currently have no shared token/ownership checks; the required Listings table is not in DB_Schema.txt.

Evidence: Backend/Controllers/Listing.cs: GetListings, CreateListing, DeleteListing; DB_Schema.txt

Login collaboration

UML communication diagram • numbered messages show ordering on a network of collaborating objects.



Numbered interaction links

1 Request login. 2–3 Obtain a backend token.
4–5 Read the profile. 6 Create the local session.
7 Return the browser-safe result.

Read the numbers in order; the same collaboration is shown with a time axis in D10.

Evidence: Website/server/api.mjs: login route, forwarding call, profile projection, session insertion

Authorization matrix

Current implementation • website permissions and backend checks are shown separately.

Action	Website requirement	Backend requirement / scope
Check membership	Signed-in session	Valid app credentials + user token; no to_user
Manage admin assignments	Admin level > 0	Caller > 0; to_user required; assigned level < caller
Set locked status	Admin level ≥ 30	Caller ≥ 30; locked must be 0 or 1
List users / manage schools	Admin level ≥ 10	Caller ≥ 10; forwarding rejected
Read / edit target profile	Admin level ≥ 10	Forwarding caller ≥ 10; target eligibility checks
Delete target account	Admin level ≥ 10	to_user required; lower admin target; locked guard
Inspect / clear attempt records	Admin level ≥ 20	Caller ≥ 20; no forwarding; choose email XOR ip_addr
Reveal admin levels in directory	Admin level ≥ 30	Backend exposes level at ≥ 27 (website is stricter)
Request target sign-out	Any admin	Forwarding guard; logout can mask validation failure

A UI permission flag does not replace backend authorization; caller and target roles remain distinct.

Evidence: Website/server/admin.mjs: adminStatus; Backend/Users.cs, AppAuthJson.cs; Controllers/*

Integration status & implementation limits

These observations qualify the model; they do not represent changes to application behavior.

IMPLEMENTED

Accounts & administration

Real browser → Node → ASP.NET workflows. Shared credentials and user tokens stay in the adapter.

PREVIEW

Marketplace interactions

Sample listing array; messages and offers only update App state. No seller delivery or durable conversations.

INCOMPLETE

Listing API integration

Website has no listing route. API expects undeclared Listings table, uses ../Data.db and has no shared auth checks.

RUNTIME LIMIT

Session scope & database constraints

One Node instance; sessions lost on restart. Account FK enforcement is not explicitly enabled by Sql connections.

Future integrations should be modeled separately until their routes, schema, authorization and persistence exist.

Evidence: latest pinned repository source; see README.md for full traceability and use-case contracts

Evidence & PowerPoint alignment

Source snapshot is pinned to f91b923f4ec8f1072b3dda07070916459342d7a2.

Lecture → modeling choices

pp. 7–14
Goals, actor roles, boundaries, include / extend

pp. 15–43
Compartments, types, visibility, multiplicities

pp. 45–59
Modularity, refinement and distinct design views

p. 60
Nodes, runtime environments and artifacts

pp. 61–81
Layering, context and component architecture

Repository → model evidence

Website src/
UI components, hooks, API contracts, preview data

Website server/
Cookies, sessions, adapter, forwarding, projections

Backend Controllers/
Account, school, admin, blocked and listing routes

Backend Components/
Auth, locks, attempts, config, SQL, scheduler

DB_Schema.txt
Declared tables, uniqueness and delete rules

Editable delivery: native PowerPoint objects + JSON vector model + reproducible Python renderer + individual SVGs.

The supplied lecture provides modeling and visual guidance; the repository supplies the system-specific behavior.